

HDOC 20

Question:

Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs.

1. Find the total surface area and volume of a sphere [double sphere_area(double radius), double sphere_volume(double radius)]
2. Find the total surface area and volume of a cylinder [double cylinder_area(double radius, double height), double cylinder_volume(double radius, double height)]
3. Find the total surface area and volume of a cone [double cone_area(double radius, double height), double cone_volume(double radius, double height)]
4. Find the total surface area and volume of a cuboid [double cuboid_area(double length, double breadth, double height), double cuboid_volume(double length, double breadth, double height)]
5. Find the base area of cylinder and cone [double circular_base_area(double radius)]
6. Find the base area of cuboid [double cuboid_base_area(double length, double breadth)]
7. Calculate the mass of a solid from its density and volume [double calculate_mass(double density)]
8. Calculate the weight of a solid [double calculate_weight()]
9. Calculate the gravitational potential energy of a solid at a given elevation [double potential_energy(double elevation)]
10. Calculate the pressure exerted by a cylinder, cone, or cuboid resting on its base [double calculate_pressure(double density)]
11. Calculate the buoyant force on a completely submerged solid [double buoyant_force(double fluid_density)]
12. Determine whether a completely immersed solid sinks, floats, or remains suspended [int determine_buoyancy(double solid_density, double fluid_density)]

Task1: ALGORITHMS:

Algorithm 1 — Main Program

1. Start.
2. Display the main menu.
3. Read the user's choice.
4. If the choice is outside 1–10, display an error.
5. Use switch-case to perform the selected operation.
6. For geometry operations, ask for solid type.
7. Validate the solid type and dimensions.
8. Call the appropriate function.
9. Display the result with the correct SI unit.
10. Repeat using a do-while loop until choice 10 is selected.
11. Stop.

Algorithm 2 — Surface Area

1. Select Sphere, Cylinder, Cone, or Cuboid.
2. Read the required dimensions.
3. Repeat input if any dimension is ≤ 0 .
4. For sphere, calculate
 $4 \times \text{PI} \times r^2$.
5. For cylinder, calculate
 $2 \times \text{PI} \times r \times (r + h)$.
6. For cone:
 - Calculate $\text{sqrt}(r^2 + h^2)$ using Newton-Raphson.
 - Calculate $\text{PI} \times r \times (r + \text{slant height})$.
7. For cuboid, calculate
 $2 \times (lb + bh + lh)$.
8. Return and display the area.

Algorithm 3 — Volume

1. Select the solid.
2. Input and validate its dimensions.
3. For sphere, calculate
 $(4/3) \times \text{PI} \times r^3$.
4. For cylinder, calculate
 $\text{PI} \times r^2 \times h$.
5. For cone, calculate
 $(1/3) \times \text{PI} \times r^2 \times h$.

6. For cuboid, calculate
 $l \times b \times h$.
7. Return and display volume in m^3 .

Algorithm 4 — Base Area

1. Select Cylinder, Cone, or Cuboid.
2. If Sphere is selected, display that its ideal geometrical base area is not defined.
3. Input and validate dimensions.
4. For cylinder/cone, calculate $\pi \times r^2$.
5. For cuboid, calculate length $\times$ breadth.
6. Display the base area in m^2 .

Algorithm 5 — Mass

1. Receive density.
2. Validate that density is greater than zero.
3. Select the solid.
4. Input and validate its dimensions.
5. Calculate its volume using the appropriate geometry function.
6. Calculate:
 $\text{mass} = \text{density} \times \text{volume}$
7. Return mass in kilograms.

Algorithm 6 — Weight

1. Obtain mass by calling `calculate_mass()`.
2. Calculate:
 $\text{weight} = \text{mass} \times \text{GRAVITY}$
3. Return weight in newtons.

Algorithm 7 — Gravitational Potential Energy

1. Read elevation.
2. Ensure elevation is ≥ 0 .
3. Obtain mass using `calculate_mass()`.
4. Calculate:
 $\text{PE} = \text{mass} \times \text{GRAVITY} \times \text{elevation}$
5. Return energy in joules.

Algorithm 8 — Pressure

1. Read and validate density.
2. Select Cylinder, Cone, or Cuboid.
3. Do not permit Sphere.
4. Input and validate dimensions.

5. Calculate volume.
6. Calculate base/contact area.
7. Calculate:
 $\text{Pressure} = \text{density} \times \text{volume} \times \text{GRAVITY} / \text{contact_area}$
8. Return pressure in pascals.

Algorithm 9 — Buoyant Force

1. Read and validate fluid density.
2. Select solid.
3. Input and validate its dimensions.
4. Calculate volume.
5. Calculate:
 $\text{Buoyant force} = \text{fluid_density} \times \text{volume} \times \text{GRAVITY}$
6. Return force in newtons.

Algorithm 10 — Buoyancy Behaviour

1. Read solid density.
2. Read fluid density.
3. Ensure both are greater than zero.
4. Compare the densities.
5. If $\text{solid_density} > \text{fluid_density}$, return SINKS.
6. If $\text{solid_density} < \text{fluid_density}$, return FLOATS.
7. Otherwise, return SUSPENDED.

Algorithm 11 — Newton-Raphson Square Root

1. Receive number.
2. If $\text{number} < 0$, return -1.
3. If $\text{number} == 0$, return 0.
4. Set $\text{old_value} = \text{number}$.
5. Calculate:
 $\text{new_value} = 0.5 \times (\text{old_value} + \text{number} / \text{old_value})$
6. Calculate $\text{difference} = \text{new_value} - \text{old_value}$.
7. If difference is negative, make it positive.
8. Set $\text{old_value} = \text{new_value}$.
9. Repeat steps 5–8 while $\text{difference} \geq \text{TOLERANCE}$.
10. Return new_value .

Task2: Test case table:

TC	Operation	Input	Expected result/behaviour	Type
TC-01	Sphere Area	$r = 2$	50.2656 m^2	Valid
TC-02	Sphere Volume	$r = 2$	33.5104 m^3	Valid
TC-03	Cylinder Area	$r=2, h=5$	87.9648 m^2	Valid
TC-04	Cylinder Volume	$r=2, h=5$	62.8320 m^3	Valid
TC-05	Cone Area	$r=3, h=4$	75.3984 m^2	Valid
TC-06	Cone Volume	$r=3, h=4$	37.6992 m^3	Valid
TC-07	Cuboid Area	$l=2, b=3, h=4$	52.0000 m^2	Valid
TC-08	Cuboid Volume	$l=2, b=3, h=4$	24.0000 m^3	Valid
TC-09	Cylinder Base	$r=2$	12.5664 m^2	Valid
TC-10	Cuboid Base	$l=2, b=3$	6.0000 m^2	Valid
TC-11	Sphere Base	Sphere	Message: base area not defined	Invalid selection
TC-12	Dimension	$r=0$ then $r=2$	Error; request radius again	Boundary
TC-13	Dimension	$h=-5$ then $h=5$	Error; request height again	Invalid
TC-14	Mass	density=1000, sphere $r=1$	4188.8 kg approx.	Valid
TC-15	Density	0 then 1000	Error; request density again	Boundary
TC-16	Density	-100 then 1000	Error; request density again	Invalid
TC-17	GPE	elevation=0	0 J	Boundary valid
TC-18	GPE	elevation=-5 then 5	Error; request elevation again	Invalid
TC-19	Pressure	Sphere	Message: pressure unsupported	Invalid selection
TC-20	Buoyancy	solid=800, fluid=1000	FLOATS	Valid
TC-21	Buoyancy	solid=1200, fluid=1000	SINKS	Valid
TC-22	Buoyancy	solid=1000, fluid=1000	SUSPENDED	Boundary
TC-23	Main Menu	choice=0	Error and menu displayed again	Invalid
TC-24	Main Menu	choice=11	Error and menu displayed again	Invalid
TC-25	Solid Menu	solid=5	Error and solid requested again	Invalid
TC-26	Square Root	number=0	0	Boundary
TC-27	Square Root	number=-4	-1	Invalid

Task3: C program:

solid.h

```
#ifndef SOLID_H
#define SOLID_H
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1
```

/* geometry.c */

```
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);
```

/* physics.c */

```
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);
```

/* math_utils.c */

```
double calculate_square_root(double number);
#endif
```

math_utils.c

```
#include "solid.h"
```

```
double calculate_square_root(double number)
{
    double old_value, new_value, difference;
    if (number < 0)
        return -1;
    if (number == 0)
        return 0;
    old_value = number;
    do
    {
        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;
        if (difference < 0)
            difference = -difference;
        old_value = new_value;
    } while (difference >= TOLERANCE);
    return new_value;
}
```

geometry.c

```
#include "solid.h"
```

```
double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

double cone_area(double radius, double height)
```

```

{
    double slant_height;
    slant_height =
        calculate_square_root(radius * radius + height * height);
    return PI * radius * (radius + slant_height);
}
double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth +
        breadth * height +
        length * height);
}
double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}
double circular_base_area(double radius)
{
    return PI * radius * radius;
}
double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"
double calculate_mass(double density)
{
    int solid;
    double radius, height, length, breadth;
    double volume = 0;
    do
    {
        printf("\nSelect Solid:\n");

```



```

printf("1. Sphere\n");
printf("2. Cylinder\n");
printf("3. Cone\n");
printf("4. Cuboid\n");
printf("Enter solid choice: ");
scanf("%d", &solid);
if (solid < SPHERE || solid > CUBOID)
    printf("Error: Invalid solid choice.\n");
} while (solid < SPHERE || solid > CUBOID);
switch (solid)
{
    case SPHERE:
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)
                printf("Error: Radius must be greater than zero.\n");
        } while (radius <= 0);
        volume = sphere_volume(radius);
        break;
    case CYLINDER:
    case CONE:
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)
                printf("Error: Radius must be greater than zero.\n");
        } while (radius <= 0);
        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
            if (height <= 0)
                printf("Error: Height must be greater than zero.\n");
        } while (height <= 0);
        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
        else
            volume = cone_volume(radius, height);

```

```

        break;
    case CUBOID:
        do
        {
            printf("Enter length (m): ");
            scanf("%lf", &length);
            if (length <= 0)
                printf("Error: Length must be greater than zero.\n");
        } while (length <= 0);
        do
        {
            printf("Enter breadth (m): ");
            scanf("%lf", &breadth);

            if (breadth <= 0)
                printf("Error: Breadth must be greater than zero.\n");
        } while (breadth <= 0);
        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
            if (height <= 0)
                printf("Error: Height must be greater than zero.\n");
        } while (height <= 0);
        volume = cuboid_volume(length, breadth, height);
        break;
    }
    return density * volume;
}

double calculate_weight(void)
{
    double density, mass;
    do
    {
        printf("Enter density of solid (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Error: Density must be greater than zero.\n");
    } while (density <= 0);
    mass = calculate_mass(density);
    return mass * GRAVITY;
}

```

```

}
double potential_energy(double elevation)
{
    double density, mass;
    do
    {
        printf("Enter density of solid (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Error: Density must be greater than zero.\n");
    } while (density <= 0);
    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}
double calculate_pressure(double density)
{
    int solid;
    double radius, height, length, breadth;
    double volume = 0;
    double contact_area = 0;
    do
    {
        printf("\nSelect Solid:\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter solid choice: ");
        scanf("%d", &solid);
        if (solid == SPHERE)
            printf("Pressure for sphere is not supported.\n");
        else if (solid < CYLINDER || solid > CUBOID)
            printf("Error: Invalid solid choice.\n");
    } while (solid < CYLINDER || solid > CUBOID);
    if (solid == CYLINDER || solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)

```

```

        printf("Error: Radius must be greater than zero.\n");
    } while (radius <= 0);
    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Error: Height must be greater than zero.\n");
    } while (height <= 0);
    contact_area = circular_base_area(radius);
    if (solid == CYLINDER)
        volume = cylinder_volume(radius, height);
    else
        volume = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
    } while (length <= 0);
    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);
    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);
    contact_area = cuboid_base_area(length, breadth);
    volume = cuboid_volume(length, breadth, height);
}
return (density * volume * GRAVITY) / contact_area;
}
double buoyant_force(double fluid_density)
{
    int solid;
    double radius, height, length, breadth;

```

```

double volume = 0;
do
{
    printf("\n1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
    printf("Enter solid choice: ");
    scanf("%d", &solid);
    if (solid < SPHERE || solid > CUBOID)
        printf("Error: Invalid solid choice.\n");
} while (solid < SPHERE || solid > CUBOID);
switch (solid)
{
    case SPHERE:
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);
        volume = sphere_volume(radius);
        break;
    case CYLINDER:
    case CONE:
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);

        if (solid == CYLINDER)
            volume = cylinder_volume(radius, height);
        else
            volume = cone_volume(radius, height);
        break;
    case CUBOID:
        do
        {

```

```

        printf("Enter length (m): ");
        scanf("%lf", &length);
    } while (length <= 0);
    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);
    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
    break;
}
return fluid_density * volume * GRAVITY;
}
int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;
    else if (solid_density < fluid_density)
        return FLOATS;
    else
        return SUSPENDED;
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

int main(void)
{
    int choice, solid, result;
    double radius, height, length, breadth;
    double density, fluid_density, solid_density;
    double elevation, answer;
    do
    {

```

```

printf("\n===== SOLID CALCULATOR =====\n");
printf("1. Surface Area\n");
printf("2. Volume\n");
printf("3. Base Area\n");
printf("4. Mass\n");
printf("5. Weight\n");
printf("6. Gravitational Potential Energy\n");
printf("7. Pressure on Base\n");
printf("8. Buoyant Force\n");
printf("9. Buoyancy Behavior\n");
printf("10. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);

```

```

if (choice < 1 || choice > 10)
{
    printf("Error: Invalid menu choice.\n");
    continue;
}
switch (choice)
{
    case 1:
    case 2:
        do
        {
            printf("\n1. Sphere\n");
            printf("2. Cylinder\n");
            printf("3. Cone\n");
            printf("4. Cuboid\n");
            printf("Select solid: ");
            scanf("%d", &solid);
            if (solid < SPHERE || solid > CUBOID)
                printf("Error: Invalid solid choice.\n");
        } while (solid < SPHERE || solid > CUBOID);
        if (solid == SPHERE)
        {
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);
                if (radius <= 0)

```

```

        printf("Error: Radius must be greater than zero.\n");
    } while (radius <= 0);
    if (choice == 1)
        answer = sphere_area(radius);
    else
        answer = sphere_volume(radius);
}
else if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Error: Radius must be greater than zero.\n");
    } while (radius <= 0);
    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Error: Height must be greater than zero.\n");
    } while (height <= 0);
    if (solid == CYLINDER)
    {
        if (choice == 1)
            answer = cylinder_area(radius, height);
        else
            answer = cylinder_volume(radius, height);
    }
    else
    {
        if (choice == 1)
            answer = cone_area(radius, height);
        else
            answer = cone_volume(radius, height);
    }
}
else
{

```



```

do
{
    printf("Enter length (m): ");
    scanf("%lf", &length);
} while (length <= 0);
do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &breadth);
} while (breadth <= 0);
do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
} while (height <= 0);
if (choice == 1)
    answer = cuboid_area(length, breadth, height);
else
    answer = cuboid_volume(length, breadth, height);
}
if (choice == 1)
    printf("Surface Area = %.4lf m^2\n", answer);
else
    printf("Volume = %.4lf m^3\n", answer);

break;

```

case 3:

```

do
{
    printf("\n1. Sphere\n");
    printf("2. Cylinder\n");
    printf("3. Cone\n");
    printf("4. Cuboid\n");
    printf("Select solid: ");
    scanf("%d", &solid);
    if (solid < SPHERE || solid > CUBOID)
        printf("Error: Invalid solid choice.\n");
} while (solid < SPHERE || solid > CUBOID);
if (solid == SPHERE)

```

```

{
    printf("Base area for a sphere is not defined.\n");
    break;
}
if (solid == CYLINDER || solid == CONE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Error: Radius must be greater than zero.\n");

    } while (radius <= 0);
    answer = circular_base_area(radius);
}
else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
    } while (length <= 0);
    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    answer = cuboid_base_area(length, breadth);
}
printf("Base Area = %.4lf m^2\n", answer);
break;
case 4:
do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
    if (density <= 0)
        printf("Error: Density must be greater than zero.\n");
}

```

```

    } while (density <= 0);
    answer = calculate_mass(density);
    printf("Mass = %.4lf kg\n", answer);
    break;
case 5:
    answer = calculate_weight();
    printf("Weight = %.4lf N\n", answer);
    break;
case 6:
    do
    {
        printf("Enter elevation (m): ");
        scanf("%lf", &elevation);
        if (elevation < 0)
            printf("Error: Elevation cannot be negative.\n");
    } while (elevation < 0);
    answer = potential_energy(elevation);
    printf("Potential Energy = %.4lf J\n", answer);
    break;
case 7:
    do
    {
        printf("Enter density of solid (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Error: Density must be greater than zero.\n");
    } while (density <= 0);
    answer = calculate_pressure(density);
    printf("Pressure = %.4lf Pa\n", answer);
    break;
case 8:
    do
    {
        printf("Enter fluid density (kg/m^3): ");
        scanf("%lf", &fluid_density);
        if (fluid_density <= 0)
            printf("Error: Fluid density must be greater than zero.\n");
    } while (fluid_density <= 0);
    answer = buoyant_force(fluid_density);
    printf("Buoyant Force = %.4lf N\n", answer);
    break;

```

```

case 9:
    do
    {
        printf("Enter solid density (kg/m^3): ");
        scanf("%lf", &solid_density);
        if (solid_density <= 0)
            printf("Error: Solid density must be greater than zero.\n");
    } while (solid_density <= 0);
    do
    {
        printf("Enter fluid density (kg/m^3): ");
        scanf("%lf", &fluid_density);

        if (fluid_density <= 0)
            printf("Error: Fluid density must be greater than zero.\n");
    } while (fluid_density <= 0);
    result = determine_buoyancy(solid_density,
                                fluid_density);
    if (result == FLOATS)
        printf("The solid FLOATS.\n");
    else if (result == SINKS)
        printf("The solid SINKS.\n");
    else
        printf("The solid remains SUSPENDED.\n");
    break;
case 10:
    printf("Exiting program...\n");
    break;
}
} while (choice != 10);
return 0;
}

```

Task4: Output:

```
kali@kali:~/solids_project$  
gcc -o solids main.c geometry.c physics.c math_utils.c  
kali@kali:~/solids_project$  
./solids
```

```
(kali@kali)-[~/solids_project]  
$ ./solids  
===== Geometrical Measurements and Physical Properties of Solids =====  
1. Surface area  
2. Volume  
3. Base area  
4. Mass  
5. Weight  
6. Gravitational potential energy  
7. Pressure on the base  
8. Buoyant force  
9. Buoyancy behavior  
10. Exit  
Enter your choice: 1  
Select the solid:  
1. Sphere  
2. Cylinder  
3. Cone  
4. Cuboid  
Enter your choice: 1  
Enter radius (m): 2  
Total surface area of sphere = 50.2656 m^2
```

```
(kali@kali)-[~/solids_project]  
$ ./solids  
===== Geometrical Measurements and Physical Properties of Solids =====  
1. Surface area  
2. Volume  
3. Base area  
4. Mass  
5. Weight  
6. Gravitational potential energy  
7. Pressure on the base  
8. Buoyant force  
9. Buoyancy behavior  
10. Exit  
Enter your choice: 7  
Select the solid:  
1. Cylinder  
2. Cone  
3. Cuboid  
Enter your choice: 2  
Enter radius (m): 1  
Enter height (m): 2  
Enter density (kg/m^3): 7800  
Pressure on base = 50957.7600 Pa
```

```
(kali㉿kali)-[~/solids_project]
$ ./solids
===== Geometrical Measurements and Physical Properties of Solids =====
1. Surface area
2. Volume
3. Base area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Enter your choice: 2
Select the solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter your choice: 3
Enter radius (m): 3
Enter height (m): 4
Volume of cone = 37.6992 m^3
```